Copyright © 2023. Bentham Science Publishers. All rights reserved.

Object-Oriented Programming

Krishna Kumar Mohbey[1], **Malika Acharya**[1]

[1] Computer Science, Central University of Rajasthan, India

Abstract

In this chapter, we explore the OOP concepts in programming that offer an effective way of making coding more easy and comprehensive. It facilitates redundancy and allows customizing the existing code.

Keywords: Namespace, Superclass and Subclass.

INTRODUCTION

Object-Oriented Programming, sometimes called OOP, organizes a computer program's components into distinct objects with similar characteristics and functions. Classes and objects are

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

the fundamental building blocks of object-oriented programming. The class serves as the blueprint, while the objects themselves are living, breathing entities capable of carrying out various operations. An object is composed of its data elements, characteristics, and behaviour, which may include actions or functions.

In procedural programming, a program is structured similar to a recipe by providing several stages, including functions and code blocks, that flow sequentially to achieve a goal. This programming paradigm is one of the most common programming paradigms. Python has always been object-oriented, much like other languages designed for general-purpose programming. It makes it possible for us to construct programs using an object-oriented methodology. Python makes it very simple to create and work with objects and classes. An object-oriented paradigm refers to building software by utilizing classes and objects. The item is connected to things in the real world, such as a computer, a house, a mobile phone, and so on. The definition of OOPS emphasizes the creation of code that may be reused. Putting together new things to use as solutions is a frequently utilized strategy. An object-oriented programming paradigm can be broken down into its core ideas, which are as follows.

CLASS AND OBJECT

A **class** is a name that can be given to a collection of objects. It is a logical entity with specific methods and unique properties. For example, a student's class should contain an attribute and a method, such as their name, age, address, and the classes they are enrolled in. The **object** functions as its independent entity, complete with a state and a set of behaviors. It may be a notebook, computer, pencil, or other things. In Python, everything is an object, and nearly anything may have both attributes and methods applied to it. When a class is defined, the class must first create an object before it may assign memory.

DATA ABSTRACTION

The practice of abstraction is a method that hides information on the system's internal workings and shows only its functionalities. Data encapsulation and data abstraction are two terms that are frequently used interchangeably. Because data encapsulation is the means through which data abstraction is achieved, it is possible to use either word interchangeably.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

ENCAPSULATION

Encapsulation is a key notion in object-oriented programming, one of today's most prevalent programming paradigms. It explains the concept of enclosing the data and the methods that operate on the data within a single unit. This limits direct accessing methods and variables, which helps avoid unintentional data change. Only the object's method can change the object's variables; this is done to prevent unintentional changes. These particular variables fall under the category of “private variables.” Encapsulation can be illustrated by how a class stores all information about its member functions, variables, and so on. The objective of information hiding is to ensure that the state of an object is always valid by regulating access to its attributes while keeping those properties hidden from the view of the outside world.

INHERITANCE

Inheritance is the most fundamental component of object-oriented programming, miming the inheritance process that occurs in real life. Inheritance is an essential component. It states that all of the characteristics and behaviors of the parent object are passed down to the child object through inheritance. Through inheritance, we can build a class capable of taking on all of the characteristics and actions of another class. The new class is considered to be a derived class or a child class, whereas the base class or the parent class is regarded as the class

whose properties are gained. It assures that the code can be utilized in different applications.

POLYMORPHISM

The term “polymorphism” originates from the combination of the words “poly” and “morphs.” The prefix poly denotes “many,” while the suffix morph refers to “shape.” The capacity to carry out a single activity in multiple guises is what we mean when discussing polymorphism. It employs a single category of items, such as a method, operator, or object, to stand in for several different types in various contexts. For example, we have a single addition operator capable of adding a wide variety of value types.

Defining a Class

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

The keyword **class**, followed by the class name, is used to build a class in Python. The following is the syntax for creating a class.

Syntax:

```
class <Class_Name>
        # data members
        # member functions
```

A class declares all its attributes in a new local namespace. Data members or functions may be included as attributes.

It also contains unique attributes that start with double underscores. For example, ***__doc__*** returns the class's docstring. The following statement can access it.

```
<class-name>. __doc__.
```

A new class object with the same name is generated when we define a class. We may use this class object to access the various attributes and create new objects of that class.

Example:

```
1.  class Course:
2.      "The Python Programming course"
3.      lec= 30
4.      def fun(self):
5.          print('Lectures on Python')
6.
7.  print (Course.lec)
8.  # call function
9.  print (Course.fun)
10. # display docstring
11. print (Course.__doc__)
```

Output:

```
30
<function Course.fun at 0x7f8d97db5da0>
The Python Programming course
```

Creating an Object

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Before we utilize a class's properties or methods in another class or function, we need to instantiate the class. Simply naming a class by its name is one way to initialize it. The following syntax is mentioned to generate a class instance (object).

Syntax:

```
<object-name> = <class-name>(<arguments>)
```

Example:

```
1.  class Course:
2.      'The Python Programming course'
3.      lec= 30
4.      enroll = 89
5.      def fun(self):
6.          print('No.of scheduled lectures', self.lec)
7.          print('Enrolled students', self.enroll)
8.
9.  detail= Course()
10. # Accessing  variables
11. print (detail.lec)
12. print(detail.enroll)
13. #Accessing function
14. print (detail.fun())
```

Output:

```
30
89
No.of scheduled lectures 30
Enrolled students 89
```

Deleting Properties or Object

Using the **del** keyword, we can delete the object's properties or the object itself. The below examples demonstrate the deletion of some properties and objects respectively.

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
1.  class Course:
2.      'The Python Programming course'
3.      lec= 30
4.      enroll = 89
5.      def fun(self):
6.          print('No.of scheduled lectures', self.lec)
7.          print('Enrolled students', self.enroll)
8.
9.  detail= Course()
10. # Accessing  variables
11. print("----------Before Deletion-----------")
12. print (detail.lec)
13. print(detail.enroll)
14. #Accessing function
15. print (detail.fun())
16.
17. del Course.lec
18. print("---------------After Deletion-------------------------------")
19. print (detail.fun())
```

Output:

```
----------Before Deletion-----------

30

89

No.of scheduled lectures 30

Enrolled students 89

---------------After Deletion-------------------------------

Traceback (most recent call last):

File "<string>", line 19, in <module>

  File "<string>", line 6, in fun

AttributeError: 'Course' object has no attribute 'lec'
```

The below example shows the deletion of an already-created object.

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
class Course:
    'The Python Programming course'
    lec= 30
    enroll = 89

    def fun(self):
        print('No.of scheduled lectures', self.lec)
        print('Enrolled students', self.enroll)

detail= Course()
# Accessing  variables
print("----------Before Deletion-----------")
print (detail.lec)
print(detail.enroll)
#Accessing function
print (detail.fun())

del detail
print("---------------After Deletion-------------------------------")
print (detail.fun())
```

Output:

```
----------Before Deletion-----------
30
89
No.of scheduled lectures 30
Enrolled students 89
None
---------------After Deletion-------------------------------
Traceback (most recent call last):
  File "<string>", line 19, in <module>
NameError: name 'detail' is not defined
```

Data Abstraction Example

The object-oriented programming paradigm places a significant emphasis on the abstraction of data. We also have the option of concealing Python data by appending a pair of underscores (__) to the beginning of the attribute that is to be concealed. After this, the attribute will no longer be accessible outside the class *via* the associated object. The idea of data abstraction is demonstrated with Python code down below:

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
1.  class Course:
2.      'The Python Programming course'
3.      lec= 30
4.      enroll = 89
5.      __year = 2023
6.      def fun(self):
7.          print('No.of scheduled lectures', self.lec)
8.          print('Enrolled students', self.enroll)
9.          print('Year', self.__year)
10.
11. detail= Course()
12. # Accessing  variables
13. print (detail.lec)
14. print(detail.enroll)
15. print (detail.year)
```

Output:

```
30

89

Traceback (most recent call last):

  File "<string>", line 15, in <module>

AttributeError: 'Course' object has no attribute 'year'
```

CONSTRUCTORS

A constructor is a particular type of function that is used to initialize the class's instance members. The constructor in C++ or Java has the same name as the class, but Python handles constructors differently. When we create an object of a class, the constructor function executes. In Python, the constructor is defined using the ***__init__()*** method. It takes the **self** keyword as a first argument, allowing access to the class's attributes and methods. Depending on the ***__init__()*** definition, we can pass any number of arguments when constructing the class object. It's mainly used to set up the class attributes. Every class must have a default constructor if we have not defined any other constructor.

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
# Creating a class book
class Course:
    #defining constructor
    def __init__(self,l,e,s):
        print ('Constructor example')
        self.lec = l
        self.enroll = e
        self.sub= s

    # creating function
    def fun(self):
        print('No.of scheduled lectures:', self.lec)
        print('Enrolled students:', self.enroll)
        print('Year:', self.sub)

# Object creation and passing arguments to constructor
d_1 = Course(30, 200, "Python")
d_2= Course(45, 150, "C++")
# Accessing class attribute and method through objects
print ('-----Values------')
d_1.fun()
d_2.fun()
```

Output:

```
Constructor example

Constructor example

-----Values------

No.of scheduled lectures: 30

Enrolled students: 200

Year: Python

No.of scheduled lectures: 45

Enrolled students: 150

Year: C++
```

When we do not want to update the value or have self as an argument, we use the non-parameterized constructor. The below code shows an example of the non-parameterized constructor.

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
class demo:
 def __init__(self):   # default constructor
  self.stat = "Joe sat on the chair."
 # function for print
 def show(self):
  print(self.stat)

# Object for the class
d1 = demo()

# calling instane
d1.show()
```

Output:

```
Joe sat on the chair.
```

Parameterized Constructor

The parameterized constructor contains many parameters in addition to the self. The parameterized constructor is demonstrated in the Python code below:

Example:

```
    class demo:
        # Parameterised constructor

        def __init__(self, arg1):
            print("This is parametrized constructor")
            self.arg1= arg1
        def show(self, adv):
            print("Joe went", self.arg1, adv)

    d1 = demo("school")
    d1.show("today")
```

Output:

```
This is parametrized constructor
Joe went school today
```

Default Constructor

The default constructor is used when no constructor is included in the class or is not declared. It defines the objects' initial values. The below example demonstrates the concept of a default constructor.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Example:

```
class test:
    chk= "undone"

    def assign(self):
        print(self.chk)
# object of class
v1 = test()
# calling the instance method
v1.assign()
```

Output:

```
undone
```

INHERITANCE IN PYTHON

Inheritance is an essential part of the object-oriented framework that we will be using. Inheritance enables us to recycle the program's code since we do not have to start from zero when creating new classes; instead, we can utilize an existing class as a foundation. The child class acquires access to all of the data members and functions described in the parent class, and it also inherits the attributes present in the parent class. A child class can have its take on how the parent class's functions should be implemented. Python may allow a derived class to inherit a base class by simply specifying the base class after the name of the derived class and surrounding it with brackets. Use the syntax indicated below in order to inherit properties and behaviours from a base class into a derived class.

Syntax:

```
class <derived-class_Name> (base-class_Name):
        # derived class body
```

A derived class can inherit properties from more than one base class. It is possible to do this form of inheritance by naming all base classes within the brackets. Inheriting properties from numerous classes may be accomplished using the syntax below:

Syntax:

```
class <derive-class_name> (<base-class 1>, <base-class 2>, ..... <base-class n>):
        # derived class body
```

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Example:

```
1.  #creating base class
2.  class student():
3.   # Function inside base class
4.   def course(self,arg1):
5.       print ('-----Base class function-------')
6.       self.arg1=arg1
7.       print ('Course name:', self.arg1)
8.
9.  # Inherited class
10. class enroll(student):
11.      # Function inside derived class
12.     def is_enroll(self,arg2):
13.         print('--------Derived class function---------')
14.         self.arg2=arg2
15.         print("enrollment:", self.arg2)
16. enr = enroll()                              # An Object for derived class
17. print(enr.is_enroll("Lui"))
18. print(enr.course("python"))
```

Output:

```
--------Derived class function---------

enrollment: Lui

-----Base class function-------

Course name: pPython
```

TYPES OF INHERITANCE

There are four types of inheritance in Python, depending on the number of child and parent groups involved. It includes single, multiple, multi-level, and hierarchical inheritances.

Single Inheritance

A derived class can inherit properties from a single base class. This paves the way for code reutilization and the addition of new capabilities to pre-existing code. The principle of single inheritance is depicted here in Fig. (**1**) .

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

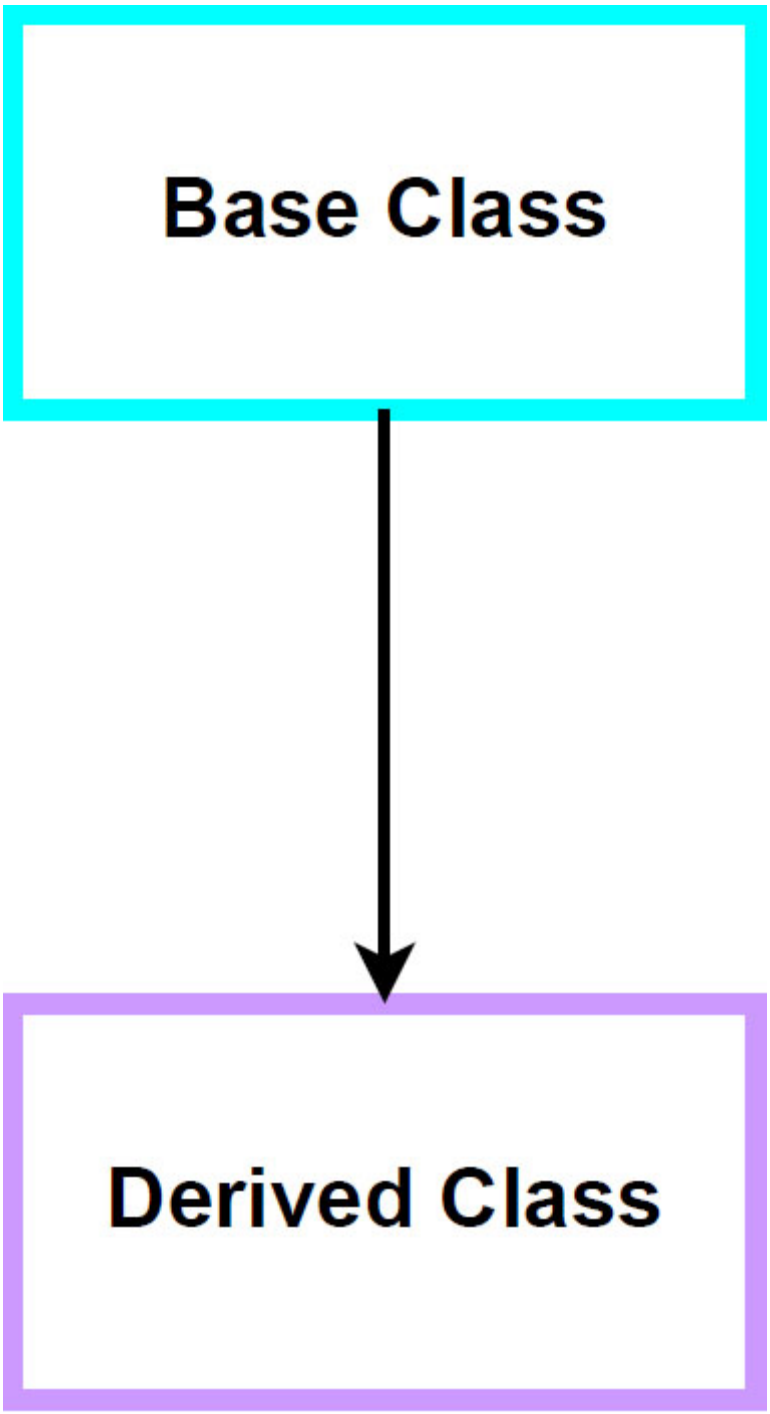


Fig. (1))

Single Inheritance.

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
#creating base class
class student():
 # Function inside base class
 def grade(self,arg1):
     print ('-----Base class function-------')
     self.arg1=arg1
     print ('grade:', self.arg1)

# Inherited class
class enroll(student):
    # Function inside derived class
   def is_pass(self,arg2):
       print('--------Derived class function---------')
       self.arg2=arg2
       print("name:", self.arg2)

enr = enroll()                                  # An Object for derived class
print(enr.grade(8))
print(enr.is_pass("Lui"))
```

Output:

```
-----Base class function-------
grade: 8
--------Derived class function---------
name: Lui
```

Multi-level Inheritance

In multi-level inheritance, characteristics of the base class and the derived classes are transmitted to the newly created derived class. There is no upper bound on the number of levels that may be archived *via* multi-level inheritance in Python; this capability is unlimited. A visual representation of the multi-level inheritance notion may be found in Fig. (2).

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

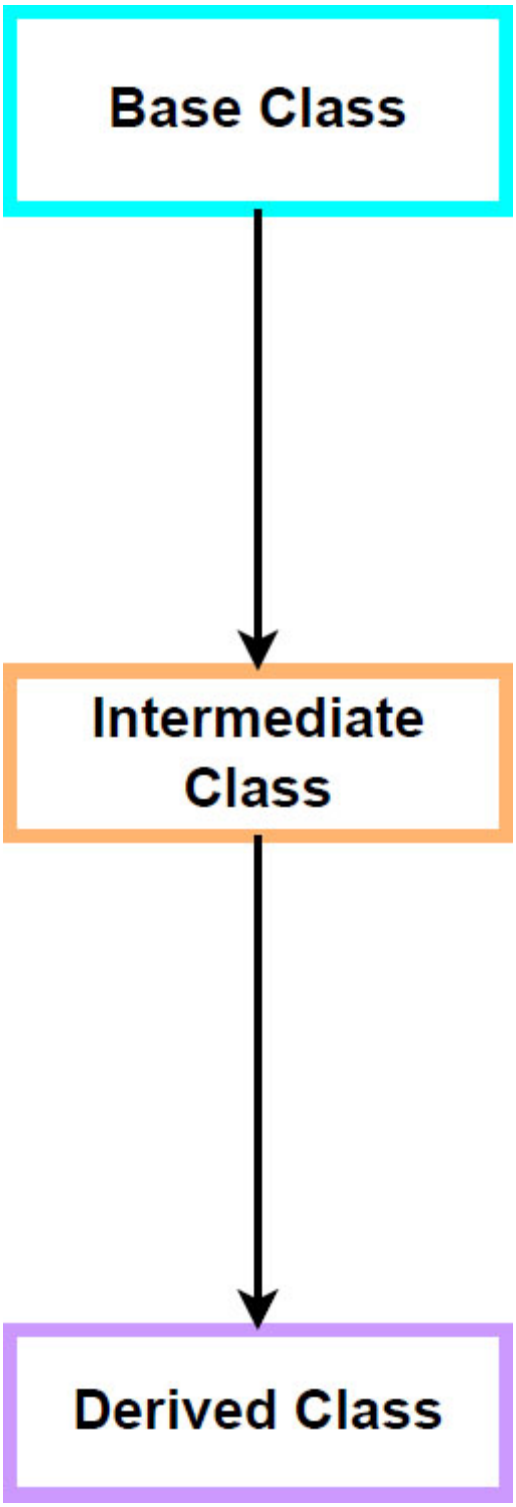


Fig. (2))

Multi-level Inheritance.

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
# Base class
class Electronics:

 def __init__(self):
     print("----Base constructor----")

 def cat1(self, category):
     print("-----Base class-----")
     self.category = category
     print('Electronic  :', self.category)

# Intermediate class
class Televisons(Electronics):
 def style_1(self, style):
  self.style = style
  print("-----Intermediate class-----")
  print("Television :", self.style)

# Derived class
class model(Televisons):
 def model_name(self, name):
     print("-----Derived class-----")
     self.name = name
     print("model  :", self.name)

s1 = model()
s1.cat1('Portable')
s1.style_1('LCD')
s1.model_name('Samsung')
```

Output:

```
----Base constructor----
-----Base class-----
Electronic  : Portable
-----Intermediate class-----
Television : LCD
-----Derived class-----
model  : Samsung
```

Multiple Inheritance

A class can derive from numerous base classes in a multiple inheritance scenario. The derived class inherits all of the base classes' properties when there are multiple inheritances. Fig. (3) illustrates multiple inheritances, where a derived class inherits the traits of two base classes.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

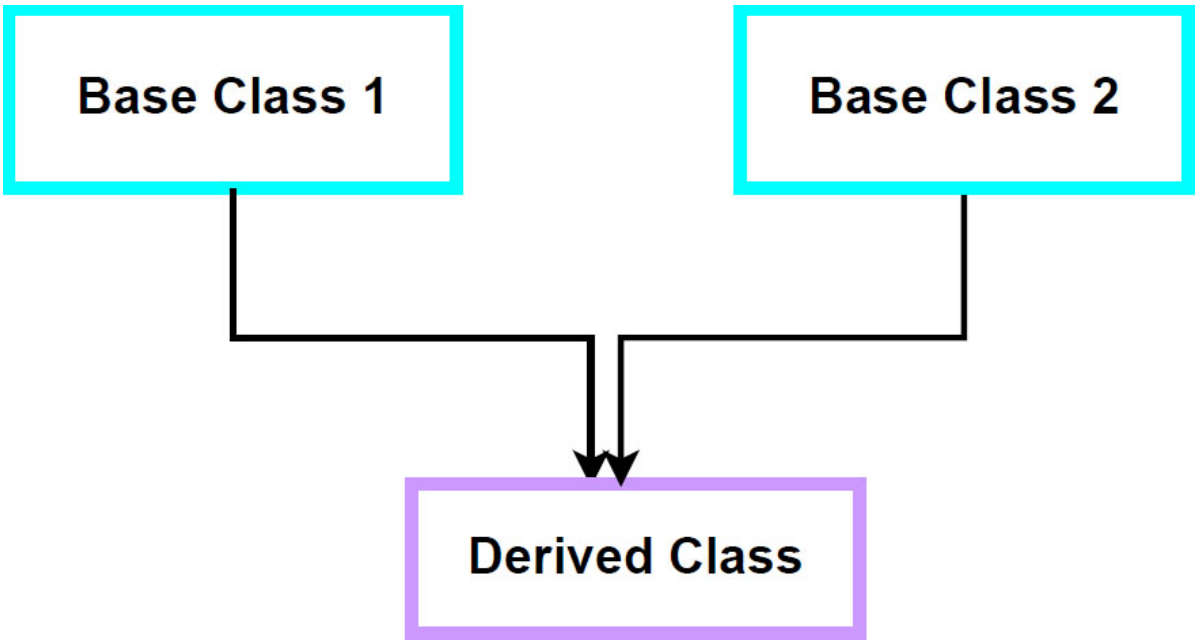


Fig. (3))

Multiple Inheritance.

Example:

```
# First Base class
class Sneakers:

 def cat1(self, category):
     print("-----Base class 1-----")
     self.category = category
     print('Sneakers  :', self.category)

# Second base class
class Sports_shoes:
 def style_1(self, style):
  self.style = style
  print("-----Base  class 2-----")
  print("Sports_shoes :", self.style)

# Derived class
class shoes(Sneakers, Sports_shoes):
 def model_name(self, name):
     print("-----Derived class-----")
     self.name = name
     print("model  :", self.name)


s1 = shoes()
s1.cat1('Spark')
s1.style_1('Bata')
s1.model_name('casuals')
```

Output:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
-----Base class 1-----
Sneakers  : Spark
-----Base  class 2-----
Sports_shoes : Bata
-----Derived class-----
model  : casuals
```

Hierarchical Inheritance

Multiple derived classes can be inherited from a single base class through a process known as hierarchy inheritance. In the example of hierarchical inheritance shown in Fig. (4) , three derived classes are descended from a single base class.

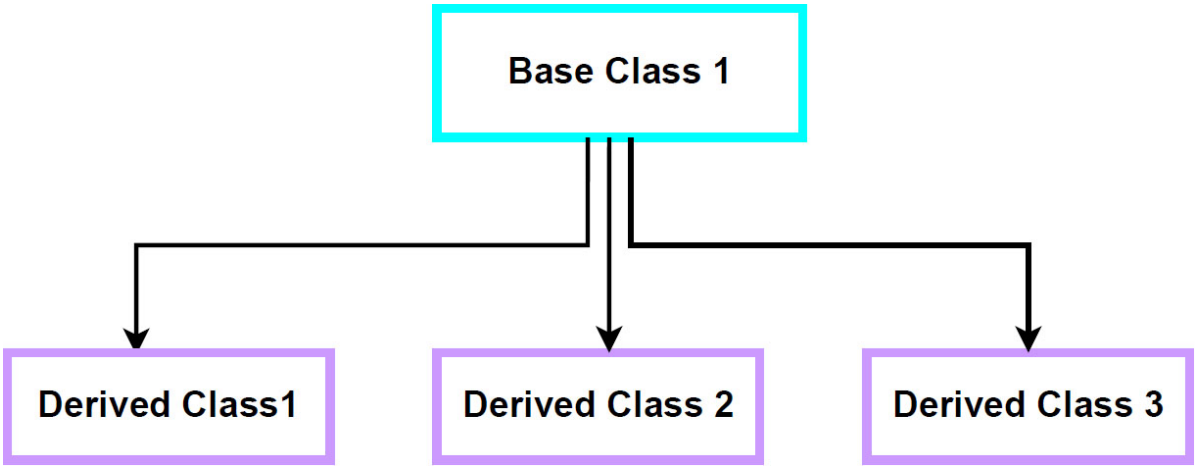


Fig. (4))

Hierarchical Inheritance.

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
# Base class
class Televisons:
 def cat1(self, category):
     print("-----Base class-----")
     self.category = category
     print('Electronic  :', self.category)

# Derived   class 1
class plasma(Televisons):
      def style_1(self, style):
       self.style = style
       print("-----Derived class 1-----")
       print("Tube :", self.style)

     # Derived class 2
     class LCD(Televisons):
      def model_name(self, name):
          print("-----Derived class 2-----")
          self.name = name
          print("model  :", self.name)

     s1 = plasma()
     s1.cat1('42 inches')
     s1.style_1('LG')


     s2 = LCD()
     s2.cat1('32 inches')
     s2.model_name('Samsung')
```

Output:

```
-----Base class-----
Electronic  : 42 inches
-----Derived class 1-----
Tube : LG
-----Base class-----
Electronic  : 32 inches
-----Derived class 2-----
model  : Samsung
```

The *issubclass(sub,sup)* method()

Use the *issubclass(sub, sup)* method to determine how the defined classes relate. It returns true if the first class is a subclass of the second; otherwise, it returns false.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Example:

```
# Base class
class Electronics:

 def cat1(self, category):
     print("-----Base class-----")
     self.category = category
     print('Electronic  :', self.category)

# Base  class 2
class Televisons():
 def style_1(self, style):
  self.style = style
  print("-----Intermediate class-----")
  print("Television :", self.style)

# Derived class
class model(Electronics, Televisons):
 def model_name(self, name):
     print("-----Derived class-----")
     self.name = name
     print("model  :", self.name)

s1 = model()
print(issubclass(model, Televisons))
print(issubclass(model, Electronics))
print(issubclass(Televisons, Electronics))
```

Output:

```
True

True

False
```

The *isinstance (obj, class)* method()

Use the *isinstance()* method to determine how objects and classes are related. It returns true if the first parameter, obj, is an instance of the second parameter, class.

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
# Base class
class Electronics:

 def cat1(self, category):
     print("-----Base class-----")
     self.category = category
     print('Electronic  :', self.category)

# Base  class 2
class Televisons():
 def style_1(self, style):
  self.style = style
  print("-----Intermediate class-----")
  print("Television :", self.style)

# Derived class
class model():
 def model_name(self, name):
     print("-----Derived class-----")
     self.name = name
     print("model  :", self.name)

s1 = model()
print(isinstance(s1, Televisons))
print(isinstance(s1, Electronics))
print(isinstance(s1, model))
```

Output:

```
False

False

True
```

POLYMORPHISM IN PYTHON

The idea of anything being able to take on various forms is referred to as polymorphism in object-oriented programming. Polymorphism allows the utilization of a single interface for inputs of a wide variety of data types and classes and varying inputs. In Python, the concept of polymorphism can be defined in several different ways.

Polymorphism in Operators

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

We know that the plus (+) operator is frequently utilized in Python programming. Despite this, there is not a single field in which it may be utilized. When working with integer data types, the plus (+) operator must carry out arithmetic addition. Similarly, string data may be concatenated by using the + operator.

Example:

```
1. a = 85
2. b = 96
3. print (a+b)
4. X = "Joe"
5. Y = "went"
6. Z= "schol"
7. print (X+" "+Y+ " "+Z)
```

Output:

```
181

Joe went schol
```

Function Polymorphism

Several Python functions may be used for many different kinds of data. A good example of this type of function is the *len()* function. Python gives the ability to work with many different sorts of data. The*len()* function is compatible with

many data types, including strings, lists, tuples, sets, and dictionaries. However, it does yield precise information about particular data types.

Example:

```
1. print (len("Programming languages"))
2. print (len([78,5588,74988]))
3. print (len({"grade": 8.02, "status": "pass"}))
4. print (len('7851'))
```

Output:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
21
3
2
4
```

Polymorphism in Class Methods

We can apply the polymorphism principle to the process of developing class methods since Python permits many classes to have methods with the same name. In this particular scenario, we might want to build a for loop that iterates over a tuple of items. Following that, we call the methods without considering the particular class type of each object. We assume that these methods are present in all classes.

Example:

```
1.  # Polymorphism in classes
2.  class A():
3.      def fun1(self):
4.          print ("python")
5.
6.      def fun2(self):
7.          print ("programming language")
8.
9.  class B():
10.     def fun1(self):
11.         print ("Anaconda")
12.
13.     def fun2(self):
14.         print ("IDE")
15.
16. obj1 = A()
17. obj2 = B()
18.
19. for i in (obj1, obj2):
20.     i.fun1()
21.     i.fun2()
```

Output:

```
python
programming language
Anaconda
IDE
```

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Method Overriding

Python's polymorphism feature enables programmers to create child classes with methods with the same names as those in their parent classes. In the inheritance process, the child class receives the methods defined for the parent class. On the other hand, a child class may change an inherited method from a parent class that the child class has inherited. This is especially helpful in situations where the inherited method from the parent class does not exactly work for the child class. In the circumstances like these, the method is re-created from scratch within the child class. The process of re-implementing a method in a parent class within a child class is referred to as method overriding or polymorphism in inheritance.

Example:

```
1.  # Example of method overriding
2.  class Subjects:
3.      def func1(self):
4.          print ('It is Base Class')
5.      def func2(self):
6.          print (' Subjects Class')
7.
8.  class Core(Subjects):
9.
10.     def func2(self):
11.         print ('Core subject class')
12.
13. class Elec(Subjects):
14.
15.     def func2(self):
16.         print ('Elec subject class')
17.
18. a1 = Subjects()
19. a1.func1()
20. a1.func2()
21.
22. print('------------')
23. b1 = Core()
24. b1.func1()
25. b1.func2()
26. print ('------------')
27. e1 = Elec()
28. e1.func1()
29. e1.func2()
```

Output:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
It is Base Class
Subjects Class
----------
It is Base Class
Core subject class
----------
It is Base Class
Elec subject class
```

Built-in Class Functions

The following are the class's built-in functions, as shown in Table 1.

Example:

```
1.  class student:
2.      def __init__(self, name, ids, sub):
3.          self.name = name
4.          self.ids = ids
5.          self.sub = sub
6.
7.  #  object of the class student
8.  s1= student("Joe",408, "Java" )
9.
10. # printing attribute name of the object s1
11. print (getattr(s1, 'name'))
12.
13. # Changing the value of attribute sub
14. setattr(s1, "sub", "Python")
15.
16. # Display modified value of sub
17. print (getattr(s1, 'sub'))
18.
19. # prints true if the student contains the attribute with  id
20. print (hasattr(s1, 'ids'))
21.
22. print("------Before deletion------")
23. print (s1.ids)
24. # deleting one attribute
25. delattr(s1, 'ids')
26.  # It will show an error since the attribute id has been deleted
27. print("------After deletion------")
28. print (s1.ids)
```

Table 1 **Built-in class functions.**

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

getattr(obj,name,default)	It is used to get the object's attribute.
setattr(obj, name,value)	It is used to give an object's attribute a specific value.
delattr(obj, name)	It deletes a specific attribute.
hasattr(obj, name)	If the object has a particular attribute, it returns true.

Output:

```
Joe
Python
True
------Before deletion------
408
------After deletion------
Traceback (most recent call last):
  File "<string>", line 28, in <module>
AttributeError: 'student' object has no attribute 'ids'
```

Built-in Class Attributes

In addition to other attributes, a Python class has several built-in class attributes that provide information about the class. The built-in class attributes are shown in Table 2.

Table 2 **Built-in class attributes.**

__dict__	It provides a dictionary with class namespace details.
__doc__	It has a string with the documentation for the class.
__name__	It is used to get the name of the class.
__module__	It allows to get to the module where this class is defined.
__bases__	It has a tuple that includes all base classes.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Example:

```
1.  class student:
2.      def __init__(self, name, ids, sub):
3.          self.name = name
4.          self.ids = ids
5.          self.sub = sub
6.
7.  def display(self):
8.          print (self.name)
9.          print (self.sub)
10. a1 = student("Akarck",4874, "Python")
11. print (a1.__doc__)
12. print (a1.__dict__)
13. print (a1.__module__)
14. print (display.__name__)
15. print (student.__bases__)
```

Output:

```
None
{'name': 'Akarck', 'ids': 4874, 'sub': 'Python'}
__main__
display
(<class 'object'>,)
```

Static Variables

All of the objects share static variables. It is also sometimes referred to as a class variable. Variables that are not static can change depending on the object they are associated with. Instance variables or non-static variables are variables that have values allocated to them within methods. In contrast, class variables are variables that have values assigned to them at the time of the class declaration.

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
# Class declaration
class Televison:
    cost = 150000              # Class or static variable
    def __init__(self,company,style):
        self.company = company      # non-static Variable
        self.style = style

# Objects creation
a1 = Televison("LG",'LCD')
a2=  Televison("Samsung",'Plasma')

print (a1.cost)  # prints 150000
print (a2.cost)  # prints 150000
print (a1.company)    # prints LG
print (a2.company)    # prints  Samsung
print (a1.style)   # prints LCD
print (a2.style)   # prints Plasma

# Class variables can be accessed using class name also
print (Televison.cost)    # prints 150000

# we change the fees for just s1 it won't be changed for s2
a1.cost = 125000
print (a1.cost)     # prints 125000
print (a2.cost)     # prints 150000

# To change the fees for all we can change them directly from the class
Televison.cost = 140000

print (a2.cost)     # prints 140000
```

Output:

```
150000

150000

LG

Samsung

LCD

Plasma

150000

125000

150000

140000
```

CONCLUDING REMARKS

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

In this chapter, we discussed object-oriented programming concepts and their implementation in Python. We discussed constructors, inheritance, polymorphism, *etc*. Various built-in class functions and attributes are described with the help of examples. Based on these examples, you can write your code for object-oriented programming. In the next chapter, we will discuss machine learning concepts and their implementations in Python.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:36.